CASE STUDY

Operations Intelligence Agent

How a one-year-old demo project became a working AI system that reads a company's supply-chain data, checks it against a live network of warehouses and depots, and writes a plain-English risk briefing — built end to end, bugs and all.

Prepared for **Gurinder Singh Ghuman**
Repository: **bigquery-defense-logistics-y42**
August 2026

PART ONE

The Idea

This project is a portfolio piece, but it's built the way a real system gets built: start with something imperfect, find out what's actually wrong with it, fix the real problems, and only then add new capability.

Where it came from

About a year before this phase of work began, this repository was built as a demo project for a job application — a BigQuery-based platform that tracks trade between countries and scores each one for supply-chain risk. It sat mostly untouched after that. When work resumed on it, the goal changed: turn it into a flagship portfolio piece called the **Operations Intelligence Agent** — something that shows, concretely, what it looks like to take a data platform and wrap a working AI agent around it.

There's a deliberate personal thread running through this project too. Its builder spent 19 years in military intelligence before moving into data and AI work, and the throughline is genuine, not decorative: military intelligence is fundamentally about pulling information from multiple, imperfect sources, cross-checking it, and producing an assessment someone can act on — without overstating what you actually know. That is, almost exactly, what this system does with supply-chain data instead of battlefield data.

> **IN ONE SENTENCE**
>
> The system looks at a country's trade risk (from one database) and a depot's supply capacity (from a completely different database), and — only when both pieces of evidence actually support it — flags the cases where the two problems are compounding each other, the same way an analyst would connect two separate intelligence reports into one assessment.

Why this is a good project to showcase

Plenty of demo projects show a chatbot answering questions about one dataset. This one deliberately does more, because that's what makes it a credible answer to "have you actually built AI agents, or just called an API": it reads from two structurally different systems (a data warehouse and a graph database), it uses a team of AI agents that each do one job instead of one model doing everything, it has an actual test suite that catches regressions, and it's wired up so it can be plugged into other AI tools (like Claude Desktop) instead of being locked into one custom app.

PART TWO

What It Actually Does

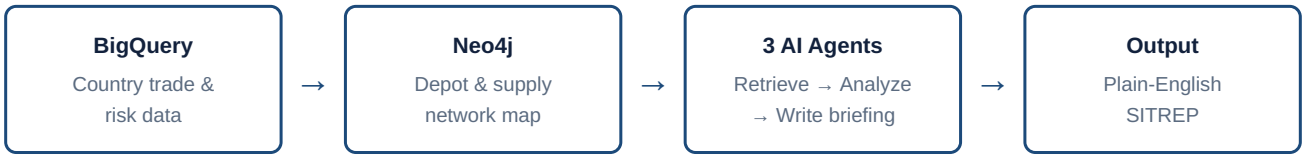
Ask it about a country — say, South Korea — and it gives you back a short, three-part briefing, written like an intelligence report.

A walk-through of one real request

Send the system a request for "KOR" (South Korea), and here's what happens, in plain terms:

1. **It looks things up.** It checks a database of country risk scores (built from simulated news events and trade data) and separately checks a network map of physical supply depots to see how each one is holding up against the demand placed on it.
2. **It thinks about what it found.** A second AI agent reads both sets of facts side by side and looks specifically for a pattern: does a country that looks "fine" on paper actually have a depot that's overwhelmed underneath? It's told, explicitly, not to say anything the numbers don't back up.
3. **It writes it up.** A third agent turns that analysis into a short brief: *Situation* (what the data shows), *Assessment* (what that means, with honest caveats about how reliable the underlying model is), and *Recommendation* (what to do about it).

For the real KOR test case, the system correctly caught something a human skimming two separate spreadsheets might easily miss: South Korea's *country-level* risk score was LOW — nothing alarming — but its *depot* (a forward supply base at Osan) was running at 189% of its average daily capacity and 268% at peak. Low national risk, but a real, specific bottleneck underneath it. That's the exact kind of compounding signal the system was built to surface.



How you can actually reach it

The system isn't locked into one interface. The same underlying logic is reachable four different ways, which was a deliberate choice, not redundancy for its own sake:

- A web API you can call from any program (built with FastAPI).
- A plug-in for Claude Desktop and other AI assistants (via a protocol called MCP — more on that below).
- A live map you open in a browser and click around on.
- An automated test suite that calls it the same way a real user would, and grades the answers.

PART THREE

How It Was Built, Step by Step

The work happened in six phases, roughly in the order below. Each phase includes not just what was built, but the real problems that came up and how they were actually solved — because that's usually the more useful part of the story.

Phase 0 Auditing before touching anything

The instinct with a year-old demo project is often to throw it out and start fresh. Instead, the first step was a full audit: go through every piece of the existing pipeline and honestly grade it — what's solid, what's fake, what's just marketing copy dressed up as documentation.

WHAT THE AUDIT FOUND

The database schema and the core business logic were genuinely well-designed and worth keeping. But several things were broken or fabricated: a "30-day" risk window that actually counted events from all time; two tables that were defined but never had any data loaded into them; a model performance number (R^2 = 0.62) quoted in the README that turned out to be **completely unreproducible** — the query needed to check it had been commented out and never run again; and event data that was randomly generated but with country sentiment hard-coded — Syria and Afghanistan were always scored negative, Norway and Denmark always positive, regardless of what "happened." That's not measurement, that's writing the conclusion first.

THE DECISION

Extend, don't rebuild. The parts that were solid would have come out identical if rebuilt from scratch, so redoing them would have been wasted effort. The five real problems got fixed directly, and two files that were 95–100% vendor marketing copy with zero technical content got deleted outright rather than "fixed."

Phase 1 Fixing the foundation

Before adding any AI on top, the underlying data pipeline had to actually produce trustworthy numbers. This meant running it for real for the first time ever — which immediately surfaced bugs that had never been caught, because nobody had ever pressed "go" on the whole thing before.

WHAT BROKE ON FIRST REAL RUN

Three separate bugs, all of the "looked fine on paper, failed the moment it touched a real database" variety: a math operator that doesn't exist in BigQuery's SQL dialect was used in two places; a data-loading step tried to insert 12 columns' worth of data into a table expecting only 7; and several blank/empty values were being interpreted as the wrong data type, which BigQuery rejected outright.

HOW IT WAS RESOLVED

Each was a small, precise fix once identified — the value of actually running a pipeline instead of just reading it. Beyond the bugs, the random data generation was replaced with a deterministic method (a hash function instead of a random number generator), so the exact same input data gets produced every time the pipeline runs — a requirement for any result to be independently checkable. The hidden evaluation query was un-commented and actually run, producing a real, verifiable model score: **$R^2 = 0.5464$** on a held-out 20% of the data the model never saw during training (the earlier 0.62 figure was retired).

Phase 2 Mapping the physical supply network

The BigQuery side answers "is this country risky?" — but it says nothing about whether the physical warehouses and supply depots resupplying that country can actually keep up with demand. That required a second kind of database entirely: a **graph database** (Neo4j), which is built for modeling networks and connections rather than rows and columns.

Eight depots were modeled (reusing the same eight countries as the trade data, so the two systems describe one coherent network from two angles), connected by 56 possible shipping routes, with roughly 2,160 simulated resupply requests spread across 90 days — again, all generated deterministically rather than randomly, for the same reproducibility reason as Phase 1.

A CALIBRATION BUG WORTH MENTIONING

The first version of this data had every single depot come back "over capacity" — which sounds dramatic but was actually a red flag that something was wrong with the setup, not the finding. The depot capacity numbers had been picked narratively ("this one should feel strained") without ever checking them against the actual demand the simulation was generating. Once checked, the capacities were recalibrated to produce a believable, mixed picture — some depots comfortably within capacity, forward bases increasingly strained — which is a much more useful and more honest signal than "everything is on fire."

Phase 3 Building the AI reasoning layer

With two solid, independent data sources in place, this is the phase that turns "two databases" into an actual *agent*. The design uses three specialized AI agents chained together, using a framework called LangGraph, rather than one AI model trying to do everything at once — the same reason a real intelligence team has researchers, analysts, and briefers as different roles instead of one person doing it all.

- **The Retriever** — a non-AI, rule-based step. Its only job is to go fetch the raw facts from BigQuery and Neo4j. It doesn't interpret anything; it just gathers evidence, the same way you wouldn't want an analyst also being the one who decides what evidence to collect.
- **The Analyst** — an AI model that reads only what the Retriever found and reasons over it, specifically looking for the cross-system pattern described earlier. It's explicitly instructed to say "no data" rather than invent an answer if something's missing.
- **The Briefing agent** — takes the Analyst's reasoning and formats it into the final Situation / Assessment / Recommendation report a person actually reads.

The guardrail: don't let it make things up

A system like this has one obvious failure mode: the AI model "remembers" something from its training and states it as fact, even when that's not what today's actual data shows. To guard against that, the system uses **hybrid retrieval** — alongside the live database facts, it also keeps a searchable local copy of the project's own methodology documents (what does "over capacity" actually mean? how reliable is the risk model, really?). When the AI needs to caveat a claim, it's instructed to pull that caveat from the actual document, not recall it from memory. This turned out to matter more than expected — see the "R^2 leak" story two sections down.

> **REAL BUGS HIT GETTING THIS RUNNING**
>
> Several: the AI model rejected a "temperature" setting the code was passing it (a parameter that controls how creative vs. literal a response is) because that particular model version had deprecated it; responses were getting cut off mid-sentence because a length limit was set too low; and the AI would sometimes return a list of sources as one run-on sentence instead of a proper list, which broke the code expecting a structured format.

> **HOW IT WAS RESOLVED**
>
> The temperature setting was removed entirely; the length limit was raised and a "please be more concise and try again" retry was added for the rare case a response still didn't fit; and a small piece of defensive code was added to accept either a list or a comma-separated sentence and convert it to the right format automatically, since AI output format can't be guaranteed 100% of the time.

Once all of that was working, the system was tested against the real KOR case described in Part Two and produced the correct, cross-referenced finding on the first fully working run.

Phase 4 Testing the AI, and watching it work

An AI system that "worked once" isn't the same as an AI system you can trust. This phase built two things: a repeatable test suite, and a way to see exactly what the AI did on every single request.

The test suite (evals)

Ten test cases were built — the eight countries with a real depot, one request that only specifies a depot ID, and one country with no depot data at all (to make sure the system correctly says "no data" instead of inventing a depot). Critically, the expected answers aren't hard-coded numbers sitting in a file that could go stale — the test suite fetches the live, current correct answer straight from the same databases the AI uses, every time it runs. On top of checking the facts are right, a second AI model acts as a **judge**, independently reading the brief and flagging anything stated that isn't actually backed by the retrieved evidence.

> **THE MOST VALUABLE BUG THE TEST SUITE EVER CAUGHT**
>
> Not a crash — a subtler, more dangerous failure. One test case asked about a depot directly, without asking about its country's risk score at all. The AI's written brief still confidently cited "R^2 is about 0.55" — a real number, just entirely irrelevant to this specific request, because that data was never even retrieved for this case. The judge model caught it. Digging into *why* it happened turned up the real root cause: the instructions given to the Analyst agent, in the part meant to warn it against fabricating things, used that exact number as an *example* of good behavior — and the model was quietly treating the example itself as a fact it already knew, instead of an illustration. The fix was to rewrite the instructions so any example number is clearly hypothetical, and to add an explicit rule: don't state anything you "know" about this project generally — only what's actually in front of you for this specific request.

This is a genuinely useful lesson for anyone building AI systems: an instruction that says "don't make things up" can itself become the source of the fabrication, if the example inside that instruction contains a real, specific fact. It took three test runs (9/10, then a different failure at 8/10, then a clean 10/10) to fully work through this and one other formatting bug before the suite passed cleanly.

Watching it work (tracing)

A tool called LangSmith was wired in — with no code changes at all, just a couple of settings — so that every single request the system handles gets recorded as a step-by-step trace: which agent ran when, what it retrieved, how many tokens it used, and how long each step took. This is the difference between "it seems to work" and being able to actually show, concretely, what happened on any specific request.

Phase 5 Making it plug into anything

Up to this point, the only way to use the system was to call its web API directly. This phase wrapped the exact same underlying logic — no duplicated code — in a standard called **MCP** (Model Context Protocol), which is how AI assistants like Claude Desktop discover and call outside tools. The practical effect: once installed, you can just ask Claude Desktop a plain-English question about South Korea's supply risk, and it calls this system directly, live, to answer it — the same way it might check the weather or search a calendar.

TWO REAL GOTCHAS

First, the exact code library needed for this had recently renamed one of its core building blocks (from "FastMCP" to "MCPServer") and moved where it lived in the package — every tutorial and reference material still described the old name, which simply didn't exist anymore in the installed version. This was only caught by directly inspecting the installed package rather than trusting the documentation. Second, after installing it into Claude Desktop, it failed to start with a "module not found" error — Claude Desktop's config file has a setting that looks like it should tell the tool "run from this folder," but that setting is silently ignored in practice.

HOW IT WAS RESOLVED

The import path was corrected to match the actual installed library. For the config issue, the fix was to stop relying on the "working folder" setting and instead pass the folder location as an explicit environment variable — which Claude Desktop does respect. After that, it started cleanly and was confirmed working live, called from a completely separate AI session, proving it genuinely works for any MCP client, not just the one it happened to be built and tested against.

Phase 6 Making it visible

The last piece was a way to actually *see* the whole picture at once, rather than asking about one country or depot at a time. A self-contained map page was built (using an open-source mapping library called Leaflet, no app installation required — it just opens in a browser) that plots every country's risk level and every depot's capacity status on the same world map, color-coded, with click-for-detail popups. It's a small addition on top of everything else, but it's the first time the whole system's output is visible at a glance instead of one request at a time — and a natural home for future interactivity, like "what if we rerouted supply away from this depot," further down the roadmap.

PART FOUR

Verified Results

Every number below was actually run and checked — not copied from an earlier draft, not estimated. That distinction is the whole point of Phase 1.

Risk model performance

Metric	Value	What it means, plainly
R^2 (held-out)	0.5464	The model explains about 55% of the variation in risk outcomes on data it never saw during training — a moderate, honest result, not an inflated one.
Mean Absolute Error	0.2631	On average, predictions are off by about a quarter of a risk-level unit.
In-sample R^2 (reference)	0.5365	Nearly identical to the held-out score above — a good sign the model isn't just "memorizing" its training data.

Country risk distribution (31 countries, real data)

Risk Level	Countries	Avg. Events (30d)	Avg. Sentiment
MEDIUM	15	90.0	-0.35
LOW	16	90.0	0.32
HIGH	0	—	—

The AI agent test suite

Run	Result	What was found & fixed
1st run	9 / 10	One briefing was cut off mid-response; response length limit raised.
2nd run	8 / 10	A formatting bug, plus the "R^2 leak" fabrication bug described in Phase 4.
3rd run	10 / 10	Clean pass, no further issues found.

What's live and working today

- The full BigQuery data pipeline — reproducible end to end.
- The Neo4j supply-network graph — seeded, verified, recalibrated.

- The three-agent LangGraph briefing system — verified against a real, correctly-caught cross-system risk pattern.
- A 10/10 passing automated test suite with live tracing.
- An MCP server, installed and confirmed working inside Claude Desktop.
- A live, browser-based risk map covering all 8 depot countries.

PART FIVE

What's Next

The roadmap was deliberately sequenced — data foundation, then physical network, then AI reasoning, then testing, then accessibility, then visibility — with each phase depending on the one before it actually being solid. The two items below are the deliberate, not-yet-started next steps:

- **Real-time event streaming and alerting** — right now the system answers when asked; the next step is having it proactively flag a new compounding risk the moment the underlying data changes.
- **Role-based access and data governance** — today this is a single-user demo with no authentication; a production version would need proper access control before being exposed beyond one person.

There's also a smaller, ongoing housekeeping item: tracking the actual dollar cost per AI request (the tracing tool already reports token counts; turning that into a real cost figure is still a manual step).

This project is intentionally scoped as one flagship effort worked on at a time, rather than several half-finished ones — a discipline that maps directly onto how the system itself is built: verify each layer before building the next one on top of it.

APPENDIX

Interview Prep — Likely Questions & Strong Answers

Every answer below is drawn from something that actually happened during this build, not a hypothetical — which is exactly what makes them strong interview answers.

"Tell me about a time you had to decide whether to fix or rebuild something."

JUDGMENT · LEGACY CODE

Inherited a year-old demo project and, instead of assuming it needed a rewrite, ran a full audit first — going file by file and grading what was solid, stubbed, or outright fake. The schema design and core business logic turned out to be genuinely good and would have come out identical if rebuilt, so rebuilding them would have been wasted effort. Decided to extend, not rebuild, and fixed five specific, named problems instead — including two files that were pure vendor marketing copy, which got deleted rather than "fixed."

"How do you make sure a reported metric or result is actually trustworthy?"

RIGOR · REPRODUCIBILITY

Found a model performance number ($R^2 = 0.62$) sitting in a README with no way to reproduce it — the evaluation query behind it had been commented out, and the training data was generated with a random number generator, so even re-running it would have produced different numbers each time. Fixed both: replaced the random generation with a deterministic hash-based method so the same inputs are produced every run, then actually ran the evaluation and published the real number (0.5464, on a proper held-out test set) — and explicitly retired the old unverified figure instead of quietly leaving it next to the new one.

"How do you prevent an AI system from hallucinating or overstating what it knows?"

AI SAFETY DESIGN

Two layers. First, structural: the system pulls live facts from real databases and separately retrieves caveats from actual project documentation, rather than letting the model recall anything from training memory. Second, tested: an automated judge model checks every output against exactly what evidence was retrieved for that specific request. That test suite caught a real case where the model cited a real, correct number — but for the wrong request, because it wasn't actually retrieved that time. Root cause turned out to be that the system prompt's own example of "good behavior" contained a real, specific number, and the model treated the example as a known fact. Rewrote the guardrail instructions to make examples clearly hypothetical and added an explicit "only use what's in front of you right now" rule.

"Tell me about a tricky bug you had to debug."

DEBUGGING SQL

Running a BigQuery pipeline for the first time surfaced three separate bugs at once, each with a different signature: a modulo operator (%) that simply isn't valid syntax in that SQL dialect; an INSERT statement listing 7 columns while the SELECT feeding it produced 12, a silent mismatch; and blank values defaulting to the wrong data type, which the database rejected outright. None of these were visible from reading the code — they only showed up by actually executing it, which is the real lesson: code that's never been run isn't verified, no matter how reasonable it looks.

"Describe a situation where your assumptions turned out to be wrong."

SELF-CORRECTION DATA QUALITY

Built a supply-network simulation where every single depot came back "over capacity" — which read as a dramatic finding but was actually a warning sign. The depot capacity numbers had been picked to feel narratively right ("this base should be strained") without ever checking them against the demand the simulation was actually generating. Recalibrated the numbers against the real math, which produced a much more credible, mixed result — some depots fine, forward positions increasingly strained — and a genuinely more useful signal than uniform alarm.

"How do you approach learning a new tool or library, especially when the documentation is wrong or outdated?"

ADAPTABILITY TOOLING

While building an MCP server, every tutorial referenced a class called "FastMCP" in a specific location — but the actually-installed version of the library had renamed it and moved it elsewhere. Rather than assuming the tutorials were right and the code was broken, directly inspected the installed package to find the real, current interface, and pinned the dependency version in the project so this wouldn't silently break again for someone else running the same setup later.

"Tell me about a time infrastructure or environment issues got in your way, and how you handled it."

OPS COMMUNICATION

Working across two environments — a sandboxed cloud session and a real local machine — a git operation in the sandbox left behind a stuck lock file that the sandbox itself couldn't delete. It wasn't cleaned up clearly enough at the time, and it later silently blocked an unrelated, completely normal commit from the local machine days afterward — costing real debugging time before the actual cause was found. The fix wasn't just technical (deleting the file); it was procedural: documented the exact incident and set a standing rule to always name the exact file path of anything left behind immediately, and to prefer having the lock removed from the native environment rather than working around it remotely.

"Why did you design this as multiple AI agents instead of one?"

ARCHITECTURE TRADE-OFFS

Split the work into a fact-gathering step, an analysis step, and a writing step — deliberately mirroring how a real intelligence team separates collection from analysis from briefing. The practical benefit showed up directly during testing: instructions and guardrails could be scoped tightly to exactly what each agent needs to do, which is part of why the specific fabrication bug found in testing was traceable to one exact sentence in one agent's prompt, rather than being buried in one large, do-everything prompt.

"How do you decide what to build versus what to defer?"

PRIORITIZATION SCOPE

The project deliberately runs on a "one active flagship project at a time" rule, with a running parking lot for other ideas rather than chasing every one as it comes up. Within the project itself, the same discipline applies at a smaller scale — for example, evals and observability tooling were explicitly named as necessary but deferred to an immediate follow-up phase, rather than either skipped entirely or forced into the same phase as the initial agent build, because bolting them on after the pipeline ran once end-to-end was the more reliable sequence.

"What would you do differently, or what's still unfinished?"

SELF-AWARENESS HONESTY

Two things, stated plainly rather than hidden: the model's held-out R^2 of 0.5464 is a moderate result, not a strong one, and the system is built to say so rather than oversell it. And there's no authentication or access control on any of it yet — it's an honest, single-user demo, and that's explicitly the next real step before this could be anything other than a portfolio piece.